

Preflight Checklist

- [x] I have searched [existing requests](https://github.com/anthropics/claude-code/issues?q=is%3Aissue%20label%3Aenhancement) and this feature hasn't been requested yet
- [x] This is a single feature request (not multiple features)

Problem Statement

Claude Code's `Read` tool is all-or-nothing. When an agent needs to debug a single function in a large file, it reads the **entire file** into context — consuming thousands of tokens for content it never references.

In real-world codebases, this causes agents to hit **context compaction** during multi-file investigations, losing earlier reasoning and forcing re-reads. The agent fights its memory instead of solving the problem.

Hard data from a 90,000-line Python codebase:

Metric	Value
Files over 500 lines	55
Largest file	5,976 lines
Largest class	5,172 lines (53 methods)

A typical multi-file debugging task (tracing a call chain across 6 files) consumed **~33,200 tokens** — 16.6% of the 200k context window — just from `Read` calls. Most of those tokens were irrelevant code the agent never referenced.

Proposed Solution

Add a `mode: "map"` parameter to the `Read` tool that returns a structural map instead of file contents:

```
```json
{
 "tool": "Read",
 "input": {
 "file_path": "/path/to/large_file.py",
 "mode": "map"
 }
}
```
```

Returns structural overview with line ranges (powered by tree-sitter for multi-language support):

```
...
```

large_file.py (5,976 lines)

```
class WorkflowSession          L:25-5196 (5172 lines)
  def __init__                  L:105-210 (106 lines)
  def run                       L:212-480 (269 lines)
  def _execute_tasks           L:995-1821 (827 lines)
  def _process_item            L:4059-4439 (381 lines)
  ...
...
```

The agent reads the map (~30 lines), identifies the function it needs, then calls `Read` with `offset`/`limit` to load only that function.

****Alternatively**, a new `Map` tool:**

```
```json
{
 "tool": "Map",
 "input": {
 "file_path": "/path/to/file.py",
 "function": "_execute_tasks"
 }
}
...
```
```

Or automatic structural pre-read: when `Read` is called on a file over N lines without an `offset`, return the map first instead of the full contents.

Alternative Solutions

****What I built as a workaround**** (currently in production):

1. ****`code\_map.py`**** — 50-line Python script using `ast` module to print method-level structure with line ranges
2. ****CLAUDE.md instruction**** — "For .py files over 500 lines, run `code\_map.py` before reading"
3. ****PreToolUse hook**** — fires on `Read` for `.py` files over 500 lines without an offset, injects a reminder to use code_map first; stays silent for small files or scoped reads

This works ~90% of the time but is fragile:

- Agents sometimes forget the CLAUDE.md instruction

- Subagents don't inherit hooks
- Every project needs its own setup
- Only supports Python (my codebase is Python-only, but a built-in should use tree-sitter)

A built-in solution would work for every user, every language, every project — no configuration required.

Priority

High - Significant impact on productivity

Feature Category

File operations

Use Case Example

****Real scenario, measured:****

I asked Claude Code to trace a call chain across 6 files (~9,500 total lines):

> "Trace the full pipeline: starting from the workflow engine's `\_process\_item`, follow the control flow through the action mixin, into the orchestrator, and down to the DOM inspector that locates the target element."

****Without structural reading**** (estimated): Agent reads all 6 files fully → ~33,200 tokens consumed → 16.6% of context → likely hits compaction partway through

****With my code_map workaround**** (measured):

| File | Total lines | Lines read | Reduction |
|----------------------|-------------|------------|------------------------------------|
| `workflow_engine.py` | 5,976 | 401 | **93%** |
| `actions.py` | 1,289 | 170 | **87%** |
| `element_finder.py` | 923 | 270 | **71%** |
| `input_handler.py` | 599 | 118 | **80%** |
| `dom_inspector.py` | 446 | 370 | 17% |
| `orchestrator.py` | 263 | 263 | 0% (small file, full read correct) |

****Result: 1,892 lines consumed instead of 9,496 — 80% context reduction (~26,600 tokens saved)****

The agent completed the task successfully with plenty of context headroom, and even caught a nuance in the control flow that you only notice when the agent isn't fighting compaction.

Two simpler tests also confirmed:

- **Single large file**: 85.6% context reduction (860 lines instead of 5,976)
- **Small file (control)**: 0% overhead — system correctly read the full 203-line file with no map step

Additional Context

Why tree-sitter for a built-in solution:

My workaround uses Python's `ast` module because my codebase is 100% Python. A built-in should use tree-sitter to support all languages. Tree-sitter provides:

- Method-level structure with precise line ranges
- 200+ language support
- Incremental parsing (fast on re-reads)
- Comment identification as AST nodes

Why this matters beyond my use case:

Every Claude Code user with a codebase over ~50,000 lines has experienced this: the agent reads 3-4 large files, hits compaction, loses earlier reasoning, re-reads files, hits compaction again, and produces lower-quality results.

The current tools assume files are the atomic unit of reading. But developers think in functions, classes, and methods. The tools should match that mental model.

My **50-line** workaround reduced context consumption by 80%. A built-in solution with tree-sitter support and automatic large-file detection could deliver the same savings to every Claude Code user, in every language, out of the box.